

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" " _____ 20

г. Заведующий

кафедрой

_____ М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКЕ

(2025/2026 учебный год)

_____ Мингалеева Алина Рустамовна

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный
интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С.

(должность, ученая степень, ученое звание)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" " _____ 20

г. Заведующий

кафедрой

_____ М.А. Митрохин

**ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ**

(2025/2026 учебный год)

_____ Мингалеева Алина Рустамовна

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный
интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения _____ 1 _____ семестр _____ 2 _____

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А. _____

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики к.т.н., доцент, Карамышева Н.С. _____

(должность, ученая степень, ученое звание)

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	2	25.06.26 - 25.06.26	
2	Подбор и изучение материала по теме работы	5	26.06.26 – 26.06.26	
3	Разработка алгоритма	12	26.06.26 – 27.06.26	
4	Описание алгоритма и программы	10	27.06.26 – 27.06.26	
5	Тестирование	2	27.06.26 – 28.06.26	
6	Получение и анализ результатов	6	28.06.26 – 28.06.26	
7	Оформление отчёта	9	28.06.26– 29.06.26	
	Общий объём часов	46		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ
О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ
(2025/2026 учебный год)

Мингалеева Алина Рустамовна

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный
интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

Мингалеева А.Р. выполняла практическое задание «Шейкерная сортировка». На первоначальном этапе были изучены и проанализированы особенности алгоритма шейкерной сортировки, выбран язык программирования С и среда разработки для реализации программы. В процессе выполнения практики было разработано пользовательское меню программы, реализовано измерение времени выполнения алгоритма, проведено тестирование программы и анализ полученных результатов. Также были подготовлены файл README, структура репозитория GitHub, скриншоты работы программы и оформлен отчет по практике.

Бакалавр Мингалеева А.Р. " " 2026 г.

Руководитель Карамышева Н.С. " " 2026 г.
практики

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЗЫВ
О ПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Мингалеева Алина Рустамовна

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Наименование профиля подготовки «Прикладной искусственный
интеллект»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 1 семестр 2

Период прохождения практики с 25.06.2026 по 7.07.2026

Кафедра «Вычислительная техника»

В процессе выполнения практики Мингалеева А.Р. решала следующие задачи: разработка алгоритма шейкерной сортировки, анализ результатов сортировки, сравнение существующих методов с разработанным алгоритмом.

За период прохождения практики были освоены основные понятия и технологии программирования, изучены основные инструменты языков C/C++. И получены следующие результаты: разработан алгоритм шейкерной сортировки, получены результаты работы алгоритма, сделаны выводы эффективности работы алгоритма шейкерной сортировки. Во время выполнения работы Мингалеева А.Р. показала себя ответственным, добросовестным учеником, знающим свой предмет, имеющим представление о современном состоянии науки, владеющим современными общенаучными знаниями по информатике и вычислительной технике; программированию.

За выполнение работы Мингалеева А.Р. заслуживает оценки «_____».

Руководитель практики к.т.н., Карамышева Н.С. «_____» 2026 г.

Содержание

Введение	2
1 Постановка задачи.....	3
1.1 Достоинства алгоритма шейкерной сортировки	3
1.2 Недостатки алгоритма шейкерной сортировки	3
1.3 Типичные сценарии применения данного алгоритма	3
2 Выбор решения.....	5
3 Описание программы	6
4. Схемы программы	7
4.1 Блок-схема программы	7
5 Тестирование программы	8
6 Отладка.....	9
7 Совместная разработка	10
Заключение.....	12
Список используемой литературы.....	13
Приложение А. Результаты тестирования программы	14
Приложение Б. Листинг программы.....	16

Введение

Сортировка данных является одной из наиболее распространённых операций при обработке информации. Она применяется практически во всех областях программирования и вычислительной техники, поскольку позволяет упорядочить данные для их дальнейшего хранения, анализа и быстрого поиска.

На сегодняшний день существует большое количество алгоритмов сортировки, отличающихся сложностью реализации, временем выполнения и областью применения. Одним из таких алгоритмов является шейкерная сортировка (Cocktail Sort). Она представляет собой усовершенствованный вариант пузырьковой сортировки и отличается тем, что проход по массиву выполняется в двух направлениях: сначала слева направо, затем справа налево. Благодаря этому элементы быстрее перемещаются к своим окончательным позициям, что позволяет уменьшить количество проходов массива по сравнению с классической пузырьковой сортировкой.

Целью данной практической работы является разработка консольной программы на языке программирования C, реализующей алгоритм шейкерной сортировки массива. Программа должна обеспечивать загрузку массива из CSV-файла или генерацию случайных данных, выполнение сортировки, измерение времени работы алгоритма, подсчёт количества перестановок элементов и сохранение результата в файл.

В процессе выполнения практики были получены навыки разработки консольных приложений на языке C, работы с файлами, проведения тестирования программ, использования системы контроля версий GitHub и оформления технической документации.

1 Постановка задачи

В рамках учебной практики необходимо разработать программу, реализующую алгоритм шейкерной сортировки массива.

Программа должна обеспечивать возможность загрузки массива целых чисел из CSV-файла либо генерации случайного массива заданного размера. После получения исходных данных необходимо выполнить сортировку массива методом шейкерной сортировки, определить время выполнения алгоритма и подсчитать количество перестановок элементов.

После завершения сортировки программа должна вывести результаты на экран и сохранить отсортированный массив в CSV-файл, имя которого задаётся пользователем.

При разработке программы необходимо использовать язык программирования C и обеспечить удобный пользовательский интерфейс в виде консольного меню. Также необходимо провести тестирование программы на различных наборах данных, подготовить документацию проекта и организовать совместную разработку с использованием репозитория GitHub.

1.1 Достоинства шейкерной сортировки

- простой алгоритм реализации;
- является стабильной сортировкой, сохраняя порядок одинаковых элементов;
- выполняет проход массива одновременно в двух направлениях.

1.2 Недостатки алгоритма шейкерной сортировки

- квадратичная временная сложность $O(n^2)$;
- большое количество сравнений и перестановок элементов;
- низкая эффективность при работе с массивами большого размера;
- уступает современным алгоритмам сортировки (Quick Sort, Merge Sort, Heap Sort) по производительности.

1.3 Типичные сценарии применения данного алгоритма

- сортировка списков студентов по фамилии или среднему баллу;
- упорядочивание товаров по цене, весу или количеству;

- обработка небольших таблиц данных;
- учебные проекты по изучению алгоритмов сортировки.
- сортировка результатов измерений;

2 Выбор решения

Для реализации поставленной задачи был выбран язык программирования С и среда разработки Microsoft Visual Studio.

Язык программирования С является одним из наиболее распространённых языков программирования общего назначения. Он отличается высокой производительностью, простотой работы с памятью и предоставляет программисту широкие возможности при реализации алгоритмов обработки данных. Благодаря этим особенностям язык С широко используется при разработке системного и прикладного программного обеспечения, а также при изучении основ программирования.

В качестве среды разработки использовалась Microsoft Visual Studio. Данная среда предоставляет удобные средства редактирования исходного кода, компиляции программы, поиска ошибок и отладки приложений. Использование Visual Studio значительно упрощает процесс разработки и тестирования программ.

Для хранения исходного кода и организации совместной разработки использовался сервис GitHub. Применение системы контроля версий позволило распределить задачи между участниками бригады, фиксировать изменения программы с помощью коммитов и хранить актуальную версию проекта в удалённом репозитории.

В ходе выполнения практической работы программа была реализована в виде консольного приложения. Такой способ реализации позволяет сосредоточить внимание на работе алгоритма шейкерной сортировки, обеспечивает простоту использования программы и не требует создания графического пользовательского интерфейса.

3 Описание программы

После запуска программы пользователю отображается главное меню, позволяющее выбрать способ получения исходного массива. Пользователь может загрузить данные из CSV-файла, сгенерировать массив случайных чисел либо завершить работу программы.

При выборе первого пункта программа предлагает ввести имя CSV-файла, после чего выполняется проверка его существования и корректности содержимого. Если файл успешно открыт и данные соответствуют требуемому формату, массив загружается в память программы.

При выборе второго пункта пользователь вводит количество элементов массива, а также минимальное и максимальное значения диапазона генерации случайных чисел. После проверки введенных данных программа автоматически формирует массив заданного размера.

После получения массива на экран выводятся его элементы. Если размер массива превышает двадцать элементов, отображаются только первые и последние десять значений, что позволяет сохранить удобство работы пользователя с программой.

Далее запускается алгоритм шейкерной сортировки (Cocktail Sort). Во время выполнения сортировки производится подсчет количества перестановок элементов массива. Для оценки производительности программы также измеряется время выполнения алгоритма.

После завершения сортировки пользователю выводится отсортированный массив, количество выполненных перестановок и время работы алгоритма в секундах.

В завершение работы программа предлагает ввести имя выходного CSV-файла. Отсортированный массив сохраняется в указанный файл, после чего пользователь может продолжить работу с программой либо завершить ее выполнение.

Подробный алгоритм работы программы и функции сортировки представлен в подразделе 4.1 на рисунке 1.

Листинг программы приведен в приложении Б.

4. Схемы программы

4.1 Блок-схема программы

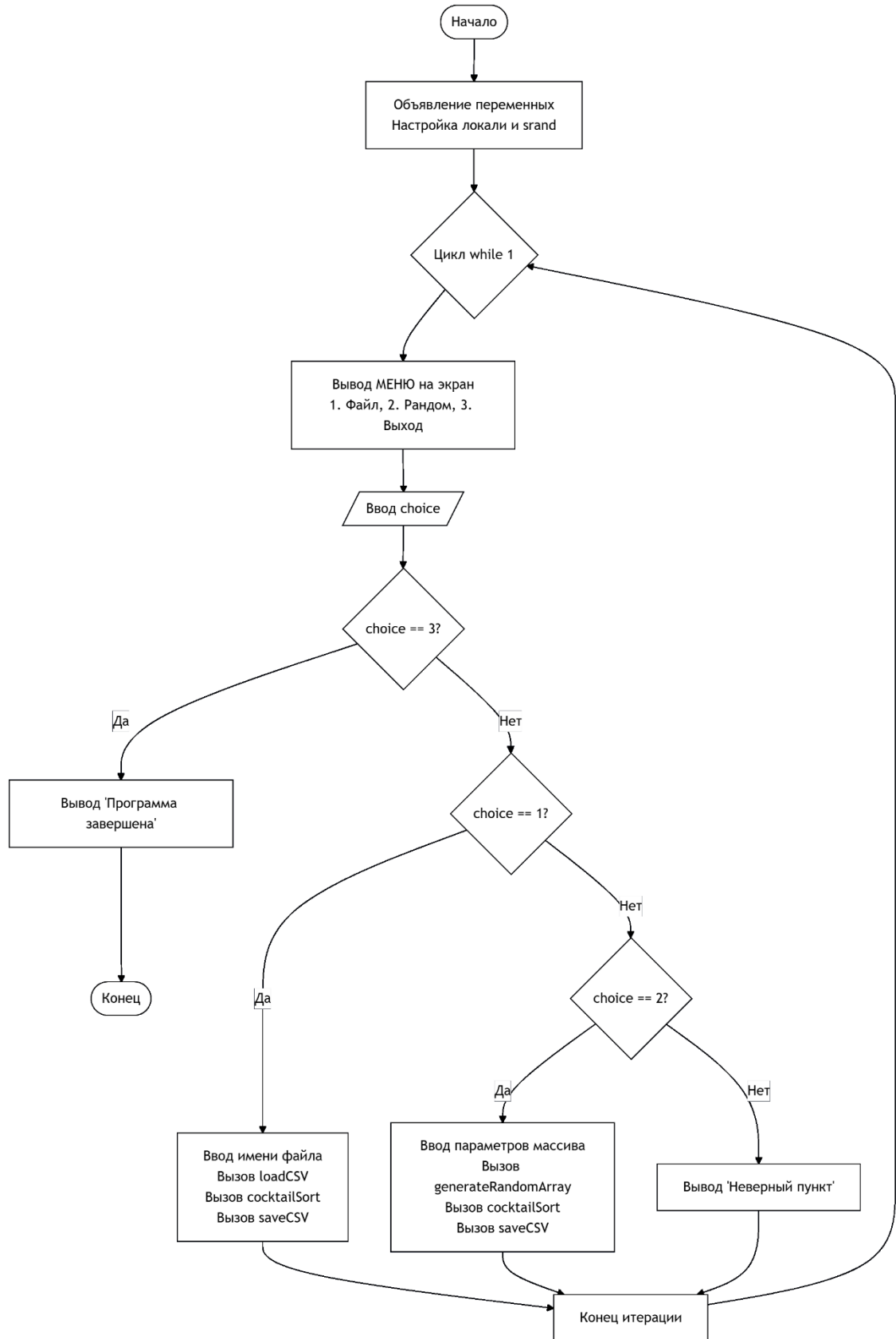


Рисунок 1 - Блок-схема программы

5 Тестирование программы

5.1 Тестирование на различных наборах данных

Для оценки работы программы было проведено тестирование алгоритма шейкерной сортировки на массивах различного размера. Во время тестирования измерялось время выполнения сортировки, а также подсчитывалось количество перестановок элементов.

Скриншоты работы программы приведены в Приложении А на рисунках А.1 – А.13.

5.2 Анализ полученных результатов тестирования

Во время тестирования программа корректно обрабатывала все варианты входных данных. При увеличении размера массива время выполнения возрастает, что соответствует квадратичной сложности алгоритма.

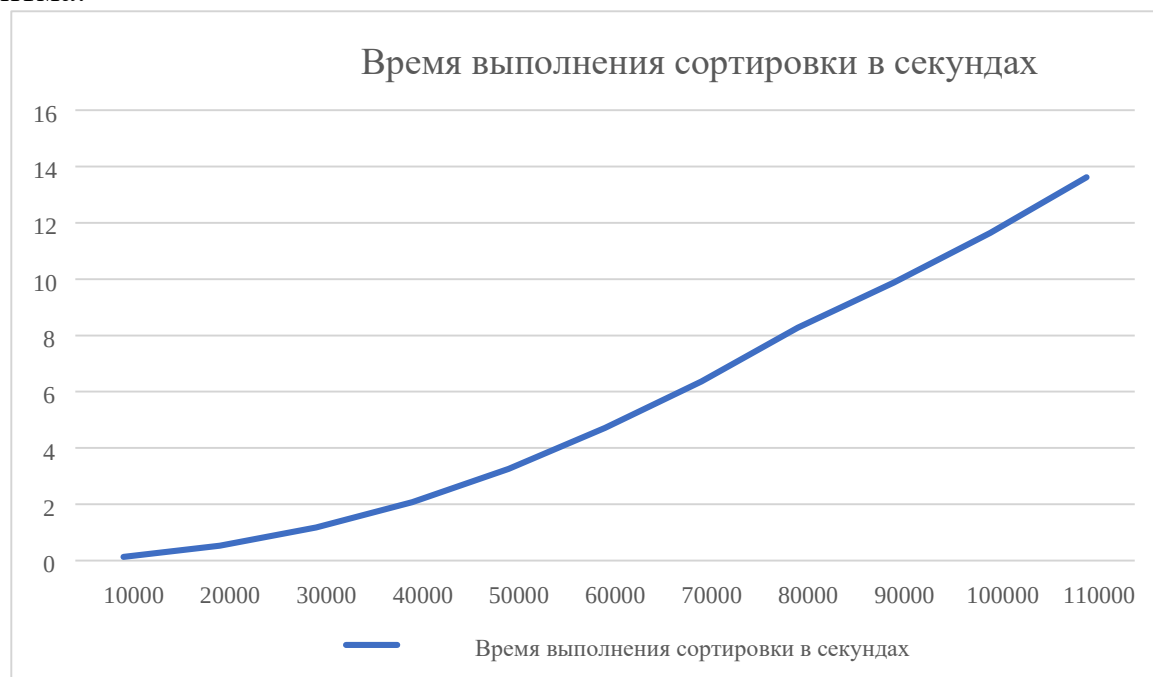


Рисунок 2 – Результаты тестирования

6 Отладка

В качестве среды разработки была выбрана программа Microsoft Visual Studio, которая предоставляет необходимые средства для написания, компиляции и отладки программ на языке C.

Для проверки корректности работы программы использовались встроенные средства отладки Visual Studio: точки останова, пошаговое выполнение программы и просмотр значений локальных переменных. Это позволило контролировать выполнение программы на каждом этапе и своевременно обнаруживать возможные ошибки.

В процессе отладки проверялась работа основных функций программы: загрузка массива из CSV-файла, генерация случайного массива, выполнение шейкерной сортировки, измерение времени работы алгоритма и сохранение результатов в файл.

Особое внимание уделялось обработке ошибочных ситуаций, таких как попытка открытия несуществующего или пустого файла, наличие некорректных данных в CSV-файле, а также ввод пользователем недопустимого размера массива. Во всех случаях программа корректно обнаруживала ошибки и выводила соответствующие сообщения.

Для анализа выполнения программы использовалась команда «Шаг с заходом» (Step Into), позволяющая последовательно выполнять инструкции программы и контролировать работу отдельных функций. В результате проведенной отладки все выявленные ошибки были устранены, после чего программа была успешно протестирована и подготовлена к эксплуатации.

7 Совместная разработка

Для удобства совместной разработки был использован сервис github. Определили задачи проекта, назначили приоритет задачам. Разделили роли, назначили исполнителей задачи.

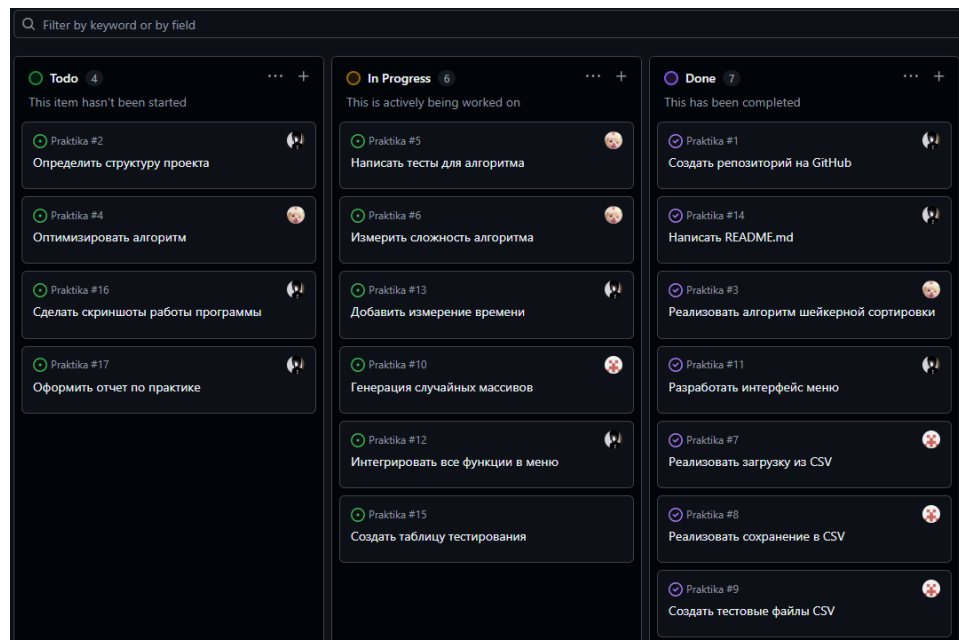


Рисунок 3 – Определение задач проекта и распределение ролей

Корректировали статус задач по мере выполнения.

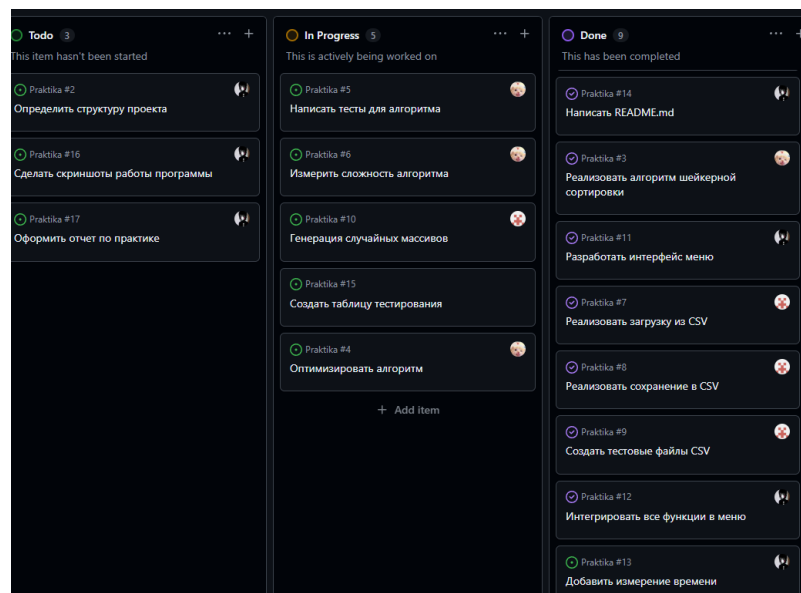


Рисунок 4 – Корректирование задач

Мною были подключены тестовые файлы и файл README, а также мною было написано меню, которое помогает ориентироваться в программе, это было зафиксировано и загружено на удаленный репозиторий Github, на ветку main.

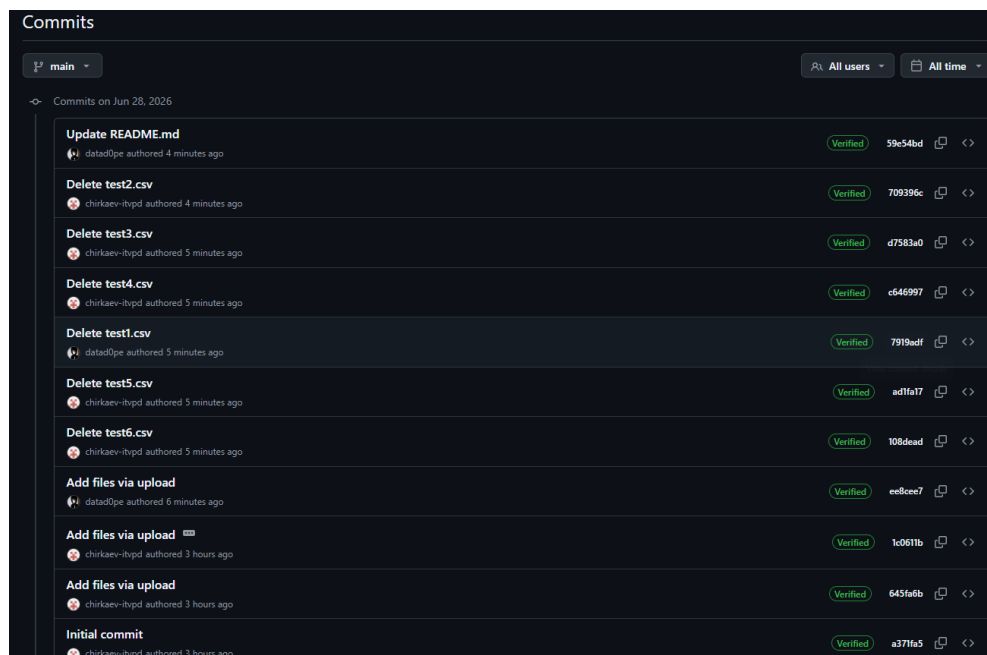


Рисунок 5 – Созданные коммиты

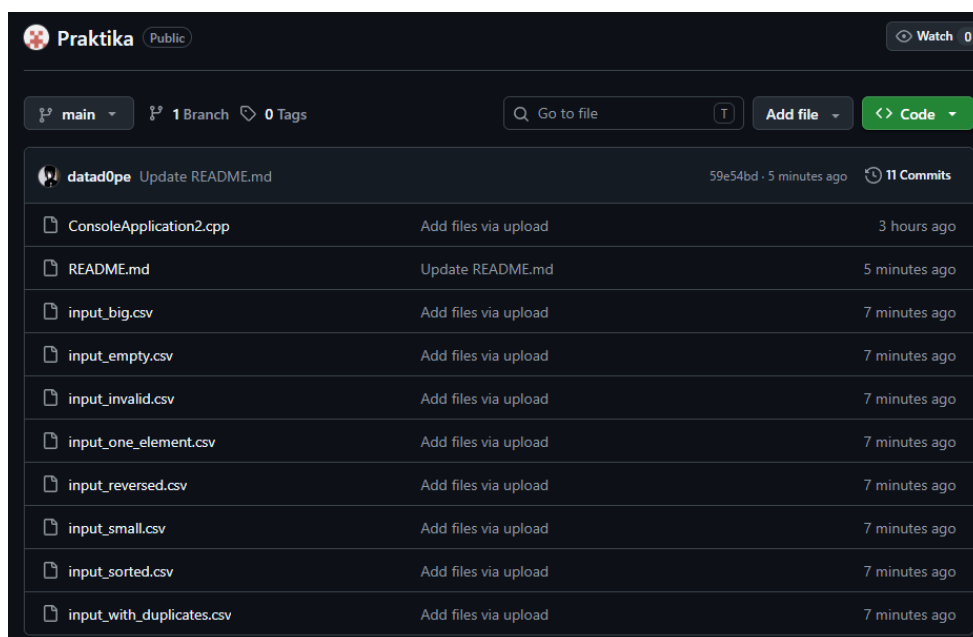


Рисунок 6– Ветка main

Ссылка на удаленный репозиторий: <https://github.com/chirkaev-itvpd/Praktika.git>

Заключение

В ходе выполнения учебной практики была разработана консольная программа, реализующая алгоритм шейкерной сортировки массива целых чисел.

Программа обеспечивает загрузку данных из CSV-файла, генерацию случайного массива, выполнение сортировки, измерение времени работы алгоритма, подсчет количества перестановок элементов и сохранение результата в выходной CSV-файл.

В рамках выполненных обязанностей было разработано пользовательское меню программы, реализовано измерение времени выполнения алгоритма, проведено тестирование программы на различных наборах данных, подготовлены таблицы результатов тестирования, оформлен файл README, организована структура репозитория GitHub и подготовлен отчет по практике.

Проведенное тестирование подтвердило корректную работу всех функций программы. Полученные результаты показали, что время выполнения алгоритма увеличивается с ростом количества элементов массива, что соответствует теоретической оценке сложности шейкерной сортировки.

В процессе выполнения практической работы были закреплены знания языка программирования C, освоены навыки работы с файлами CSV, проведено тестирование программного обеспечения, получен опыт использования системы контроля версий Git и оформления технической документации.

Список используемой литературы

1. Керниган Б., Ритчи Д. Язык программирования С. — 2-е изд. — М.: Вильямс, 2019. — 304 с.
2. Кнут Д. Э. Искусство программирования. Том 3. Сортировка и поиск. — 2-е изд. — М.: Вильямс, 2018. — 832 с.
3. Вирт Н. Алгоритмы и структуры данных. — СПб.: Питер, 2019. — 304 с.

Приложение А. Результаты тестирования программы

```
===== МЕНЮ =====  
1. Загрузить массив из CSV файла  
2. Сгенерировать массив случайных чисел  
3. Выйти из программы  
Выберите действие (1-3):
```

Рисунок А.1

```
===== МЕНЮ =====  
1. Загрузить массив из CSV файла  
2. Сгенерировать массив случайных чисел  
3. Выйти из программы  
Выберите действие (1-3): 1  
Введите имя исходного файла (например, input.csv): input_small.csv  
Массив успешно загружен из файла!  
  
Исходный массив:  
5 2 8 1 9 3 7 4 6  
  
Сортировка алгоритмом Cocktail Sort...  
Отсортированный массив:  
1 2 3 4 5 6 7 8 9  
  
-----  
Время работы алгоритма: 0,000000 секунд.  
Количество перестановок: 16  
-----  
Введите название результирующего файла для сохранения (например, output.csv):
```

Рисунок А.2

```
Введите имя исходного файла (например, input.csv): input_big.csv  
Массив успешно загружен из файла!  
  
Исходный массив:  
5 2 8 1 9 3 7 4 6 10 ... 95 92 98 91 99 93 97 94 96 100  
(Всего элементов: 1000. Вывод ограничен)  
  
Сортировка алгоритмом Cocktail Sort...  
Отсортированный массив:  
1 1 1 1 1 1 1 1 1 ... 100 100 100 100 100 100 100 100 100  
(Всего элементов: 1000. Вывод ограничен)  
  
-----  
Время работы алгоритма: 0,001000 секунд.  
Количество перестановок: 224350  
-----
```

Рисунок А.3

```
Введите имя исходного файла (например, input.csv): input_sorted.csv  
Массив успешно загружен из файла!  
  
Исходный массив:  
1 2 3 4 5 6 7 8 9 10  
  
Сортировка алгоритмом Cocktail Sort...  
Отсортированный массив:  
1 2 3 4 5 6 7 8 9 10  
  
-----  
Время работы алгоритма: 0,000000 секунд.  
Количество перестановок: 0  
-----
```

Рисунок А.4

```
Введите имя исходного файла (например, input.csv): input_reversed.csv
Массив успешно загружен из файла!

Исходный массив:
10 9 8 7 6 5 4 3 2 1

Сортировка алгоритмом Cocktail Sort...
Отсортированный массив:
1 2 3 4 5 6 7 8 9 10

-----
Время работы алгоритма: 0,000000 секунд.
Количество перестановок: 45
-----
```

Рисунок А.5

```
Введите имя исходного файла (например, input.csv): input_with_duplicates.csv
Массив успешно загружен из файла!

Исходный массив:
5 3 8 3 1 5 7 8 2 5

Сортировка алгоритмом Cocktail Sort...
Отсортированный массив:
1 2 3 3 5 5 5 7 8 8

-----
Время работы алгоритма: 0,000000 секунд.
Количество перестановок: 19
-----
```

Рисунок А.6

```
Введите имя исходного файла (например, input.csv): input_one_element.csv
Массив успешно загружен из файла!

Исходный массив:
42

Сортировка алгоритмом Cocktail Sort...
Отсортированный массив:
42

-----
Время работы алгоритма: 0,000000 секунд.
Количество перестановок: 0
-----
```

Рисунок А.7

```
Введите имя исходного файла (например, input.csv): input_invalid.csv
Ошибка: некорректные данные в файле input_invalid.csv.
```

Рисунок А.8

```
Введите имя исходного файла (например, input.csv): input_empty.csv
Ошибка: файл input_empty.csv пуст.
```

Рисунок А.9

```
Введите имя исходного файла (например, input.csv): input.csv
Ошибка: не удалось открыть файл input.csv
```

Рисунок А.10

```

===== МЕНЮ =====
1. Загрузить массив из CSV файла
2. Сгенерировать массив случайных чисел
3. Выйти из программы
Выберите действие (1-3): 2
Введите количество элементов массива (макс. 10000): 150
Введите минимальное значение диапазона: 35
Введите максимальное значение диапазона: 182
Массив случайных чисел успешно сгенерирован!

Исходный массив:
157 69 135 102 42 37 85 170 63 180 ... 141 69 137 94 143 167 171 110 172 132
(Всего элементов: 150. Вывод ограничен)

Сортировка алгоритмом Cocktail Sort...
Отсортированный массив:
35 36 36 37 38 38 39 40 40 41 ... 174 175 175 176 176 179 180 181 182 182
(Всего элементов: 150. Вывод ограничен)

-----
Время работы алгоритма: 0,000000 секунд.
Количество перестановок: 5127
-----
Введите название результирующего файла для сохранения (например, output.csv): output.csv
Результат успешно сохранен в файл output.csv

```

Рисунок А.10

```

Введите количество элементов массива (макс. 10000): 10001
Некорректный размер массива. Возврат в меню.

```

Рисунок А.11

```

Выберите действие (1-3): 4
Неверный пункт меню. Попробуйте снова.

```

Рисунок А.12

```

Выберите действие (1-3): 3
Программа завершена.

```

Рисунок А.13

Приложение Б. Листинг программы

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>
#include <time.h>
#include <windows.h>

#define MAX_SIZE 10000

long long cocktailSort(int arr[], int n)
{
    int left = 0;
    int right = n - 1;
    int swapped = 1;
    long long swap_count = 0;

    while (swapped)
    {
        swapped = 0;

```

```

    for (int i = left; i < right; i++)
    {
        if (arr[i] > arr[i + 1])
        {
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
            swapped = 1;
            swap_count++;
        }
    }

    if (!swapped)
        break;

    right--;
    swapped = 0;

    for (int i = right; i > left; i--)
    {
        if (arr[i - 1] > arr[i])
        {
            int temp = arr[i];
            arr[i] = arr[i - 1];
            arr[i - 1] = temp;
            swapped = 1;
            swap_count++;
        }
    }

    left++;
}
return swap_count;
}

int loadCSV(const char* filename, int arr[])
{
    FILE* file = fopen(filename, "r");
    if (file == NULL)
    {
        return -1;
    }

    char line[65536];

    if (fgets(line, sizeof(line), file) == NULL)
    {
        fclose(file);
        return -2;
    }

    fclose(file);

```

```

int count = 0;
char* token = strtok(line, ",");

while (token != NULL)
{
    if (count >= MAX_SIZE) break;

    char* endptr;
    long value = strtol(token, &endptr, 10);

    while (*endptr == ' ' || *endptr == '\n' || *endptr == '\r' ||
*endptr == '\t')
    {
        endptr++;
    }

    if (*endptr != '\0')
    {
        return -3;
    }

    arr[count++] = (int)value;
    token = strtok(NULL, ",");
}

return count;
}

int saveCSV(const char* filename, int arr[], int n)
{
    FILE* file = fopen(filename, "w");
    if (file == NULL)
        return 0;

    for (int i = 0; i < n; i++)
    {
        fprintf(file, "%d", arr[i]);
        if (i < n - 1)
            fprintf(file, ",");
    }

    fclose(file);
    return 1;
}

void printArray(int arr[], int n)
{
    if (n > 20) {
        for (int i = 0; i < 10; i++) printf("%d ", arr[i]);
        printf("... ");
        for (int i = n - 10; i < n; i++) printf("%d ", arr[i]);
        printf("\n(Всего элементов: %d. Вывод ограничен)\n", n);
    }
}

```

```

    }
    else {
        for (int i = 0; i < n; i++)
        {
            printf("%d ", arr[i]);
        }
        printf("\n");
    }
}

void generateRandomArray(int arr[], int n, int min_val, int max_val)
{
    for (int i = 0; i < n; i++)
    {
        arr[i] = min_val + rand() % (max_val - min_val + 1);
    }
}

int main(void)
{
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    srand((unsigned int)time(NULL));

    int arr[MAX_SIZE];
    int n = 0;
    int choice;

    char input_filename[256] = "input.csv";
    char output_filename[256] = "output.csv";

    while (1)
    {
        printf("\n===== МЕНЮ =====\n");
        printf("1. Загрузить массив из CSV файла\n");
        printf("2. Сгенерировать массив случайных чисел\n");
        printf("3. Выйти из программы\n");
        printf("Выберите действие (1-3): ");

        if (scanf("%d", &choice) != 1) {
            printf("Ошибка ввода чисел! Выход.\n");
            break;
        }

        if (choice == 3) {
            printf("Программа завершена.\n");
            break;
        }

        if (choice == 1) {

```



```

        printf("Введите имя исходного файла (например, input.csv):
");
        scanf("%255s", input_filename);

        n = loadCSV(input_filename, arr);

        if (n == -1) {
            printf("Ошибка: не удалось открыть файл %s\n",
input_filename);
            continue;
        }
        if (n == -2) {
            printf("Ошибка: файл %s пуст.\n", input_filename);
            continue;
        }
        if (n == -3) {
            printf("Ошибка: некорректные данные в файле %s.\n",
input_filename);
            continue;
        }
        printf("Массив успешно загружен из файла!\n");
    }
    else if (choice == 2) {
        int min_val, max_val;
        printf("Введите количество элементов массива (макс. %d): ",
MAX_SIZE);
        scanf("%d", &n);
        if (n <= 0 || n > MAX_SIZE) {
            printf("Некорректный размер массива. Возврат в
меню.\n");
            continue;
        }

        printf("Введите минимальное значение диапазона: ");
        scanf("%d", &min_val);
        printf("Введите максимальное значение диапазона: ");
        scanf("%d", &max_val);

        if (min_val > max_val) {
            int t = min_val; min_val = max_val; max_val = t;
        }

        generateRandomArray(arr, n, min_val, max_val);
        printf("Массив случайных чисел успешно сгенерирован!\n");
    }
    else {
        printf("Неверный пункт меню. Попробуйте снова.\n");
        continue;
    }

    printf("\nИсходный массив:\n");
    printArray(arr, n);

```

```

printf("\nСортировка алгоритмом Cocktail Sort...\n");

clock_t start_time = clock();
long long total_swaps = cocktailSort(arr, n);
clock_t end_time = clock();

double time_taken = (double)(end_time - start_time) /
CLOCKS_PER_SEC;

printf("Отсортированный массив:\n");
printArray(arr, n);

printf("\n-----\n");
printf("Время работы алгоритма: %.6f секунд.\n", time_taken);
printf("Количество перестановок: %lld\n", total_swaps);
printf("-----\n");

printf("Введите название результирующего файла для сохранения
(например, output.csv): ");
scanf("%255s", output_filename);

if (saveCSV(output_filename, arr, n)) {
    printf("Результат успешно сохранен в файл %s\n",
output_filename);
}
else {
    printf("Ошибка сохранения в файл %s\n", output_filename);
}

return 0;
}

```